

소프트웨어 개발 완료 보고서

[관극로그 : 공연 관람 가계부]

문서 버전 : v1.0

작성일 : 2026-10-01

조 : 6조

작성자 : 이재성

문서 관리

항목	내용
문서명	소프트웨어 개발 완료 보고서
프로젝트명	관극로그 : 공연 관람 가계부
문서 ID	DOC-002
문서 버전	v1.0
보고 대상	발주자 김가은
보고일	2026-10-01
보안 등급	일반

1. 개발 개요

항목	내용
개발 목적	같은 공연을 여러 번 보면 회차마다 할인, 쿠폰, 예매 수수료, 양도, MD 구매가 달라서 한 공연에 실제로 얼마를 썼는지 알기 어렵다는 발주자의 문제를 해결한다. 관람 1회마다 실제 구매금액과 순지출을 자동으로 계산하고, 월별·공연별 통계와 순위, 검색으로 지출을 한눈에 확인할 수 있게 한다.
개발 기간	2026-09-07 ~ 2026-09-29
발주자	김가은
개발 주관 부서	6조
책임자	이재성
참여 인력	개발자 1명 (이재성) : 기획, 설계, 구현, 테스트, 문서 작성 발주자 1명 (김가은) : 요구사항 제시
개발 환경	Python 3.11, macOS 터미널, 추가 라이브러리 없음, 데이터는 JSON 파일(data.json)

2. 개발 범위 및 주요 결과

구분	계획 범위	완료 결과	상태	비고
기능	FR-01 ~ FR-06 EH-01 ~ EH-05	기능 요구사항 6개와 예외 처리 5개를 모두 구현	완료	4장 참고
화면	UI-01 ~ UI-06 (설계서 6.1)	메인 메뉴, 공연 관리, 관람 기록 관리, 월별 통계, 공연별 분석·순위, 검색 화면과 저장 전 계산 결과 확인 화면 구현	완료	메뉴를 넘어갈 때마다 화면을 지우고 새로 그림
인터페이스	외부 연계 없음 (콘솔 입력, 로컬 파일)	모듈 사이 호출을 설계서 3.1 구성도와 같게 구성	완료	외부 시스템 없음
데이터	data.json (설계서 5.1 구조), 샘플 데이터 파일	data.json 저장·불러오기와 손상 파일 백업 구현. 샘플 데이터 파일(sample_data.json) 작성	완료	샘플 : 공연 5건, 관람 9건

3. 주요 개발 내용

번호	개발 항목	주요 내용	관련 산출물
1	공연·관람 기록 관리 (FR-01)	공연 등록·수정·삭제와 관람 기록 추가·수정·삭제·상세 보기. 번호(P1, V1)는 자동으로 붙이고, 공연을 지우면 그 공연의 관람 기록도 같이 지운다.	cli.py, service.py
2	지출 계산 (FR-02)	실제 구매금액 = 정가 - 정가 × 할인율 - 쿠폰 합계 + 예매 수수료, 순지출 = 실제 구매금액 + MD 합계 - 양도 금액. 쿠폰 여러 장, 수수료 면제 쿠폰, 양도받음·양도함을 처리하고, 저장 전에 계산 결과를 보여준 뒤 저장 여부(y/n)를 묻는다.	expense.py
3	통계·순위 (FR-03 ~ FR-05)	월별 합계, 공연별 누적 합계와 회당 평균, 총 순지출 순위. 순위는 금액이 같으면 같은 순위로 매기고 관람 기록이 없는 공연은 뺀다.	statistics.py
4	검색 (FR-06)	공연명·공연장·캐스팅 중 하나를 골라 대소문자 구분 없이 부분 일치로 찾는다. 캐스팅 검색은 그 배우가 나온 관람 기록만 보여준다.	service.py

번호	개발 항목	주요 내용	관련 산출물
5	입력 검증 (EH-01 ~ EH-05)	입력할 때 바로 검사(1차)하고, 저장 직전에 service.py가 한 번 더 검사(2차)한다. 양도받은 티켓에 쿠폰이 있는 것 같은 모순도 막는다.	input_utils.py, cli.py
6	데이터 저장	시작할 때 한 번 불러오고, 바뀔 때마다 전체를 저장한다. 저장할 내용을 먼저 만들어 확인한 뒤 파일을 열고, 파일이 깨졌으면 data_backup.json으로 옮겨 둔다.	repository.py, data.json
7	한글 입력 안정화	반쯤 지워진 한글(깨진 바이트)을 걸러내고 readline을 적용해서, 저장 파일이 깨지거나 화면과 입력값이 어긋나는 문제를 막았다.	main.py, input_utils.py
8	화면·표 출력	화면마다 구분선과 제목, 메뉴 이동 시 화면 지우기, 한글을 2칸으로 세서 표 줄 맞춤, 칸보다 긴 글자는 '..'으로 표시, 금액 천 단위 콤마.	cli.py
9	자체 테스트·사용 설명서	계산·검증·통계·저장을 확인하는 자체 테스트(12종, 확인 항목 104개)와 처음 쓰는 사람을 위한 README.md 작성.	test_calc.py, README.md

4. 요구사항 이행 현황

요구사항 ID	요구사항 내용	구현 결과	검증 결과	상태
FR-01	공연 및 관람 기록 관리	공연 등록·수정·삭제, 관람 기록 추가·수정·삭제·상세 보기, 번호 자동 부여	테스트 1, 8 통과	충족
FR-02	티켓 및 지출 내역 관리	실제 구매금액·순지출 자동 계산, 쿠폰 여러 장, 수수료 면제 쿠폰, 양도받음·양도함	테스트 2, 3 통과	충족
FR-03	월별 지출 통계	연월로 관람 기록을 골라 관람 횟수와 티켓·MD·순지출 합계를 표로 출력	테스트 4 통과	충족
FR-04	공연별 지출 분석	공연의 관람 기록 전체, 누적 합계, 회당 평균	테스트 5 통과	충족
FR-05	공연별 순위	총 순지출이 큰 순서, 금액이 같으면 같은 순위, 관람 기록 없는 공연 제외	자체 테스트 (순위 11개 항목) 통과	충족

요구사항 ID	요구사항 내용	구현 결과	검증 결과	상태
FR-06	공연 검색 및 필터링	공연명·공연장·캐스팅, 대소문자 구분 없는 부분 일치	자체 테스트 (검색 11개 항목) 통과	충족
EH-01	잘못된 입력 처리	글자·음수·날짜 형식·없는 날짜를 넣으면 이유를 알려주고 다시 입력받음	테스트 7 통과	충족
EH-02	필수 항목 누락 차단	공연명·관람일·티켓 정가·예매 수수료·거래 유형이 비어있으면 막고, 저장 직전에 2차 검증	자체 테스트 (2차 검증 15개 항목) 통과	충족
EH-03	데이터 부재 처리	공연·관람 기록·검색 결과가 없으면 안내 문구를 띄우고 이전 메뉴로	테스트 4 통과	충족
EH-04	삭제 확인 처리	지울 내용과 같이 지워질 관람 기록 수를 보여주고 y/n 확인	테스트 6 통과	충족
EH-05	0으로 나누기 방지	관람 횟수가 0이면 평균을 계산하지 않고 안내	테스트 5 통과	충족
비기능	직관적 화면	화면마다 구분선과 제목, 메뉴를 넘어갈 때마다 화면을 지우고 새로 그림	터미널 실행으로 확인	충족
비기능	가독성	표 정렬 (한글은 2칸으로 계산), 금액 천 단위 콤마	통계·순위 화면 확인	충족
비기능	안정성	잘못된 입력, 깨진 저장 파일, 한글 입력 문제에도 종료되지 않음	EH 시나리오 실행	충족
비기능	데이터 영속성	등록·수정·삭제 때마다 data.json에 저장	테스트 8 통과	충족
비기능	코드 관리	설계서 2.2 원칙대로 7개 파일로 분리	모듈 호출 관계 확인	충족

※ 테스트 1 ~ 8은 「기획 및 요구사항 정의서」 7장의 테스트 시나리오 번호이다.

5. 시험 및 품질 활동 결과

시험 유형	수행 기간	대상/범위	케이스 수	성공	실패	조치 상태
단위 테스트	2026-09-17 ~ 09-28	test_calc.py 12종 : 지출 계산, 입력 검사, 월별 통계, 공연별 분석, 순위, 검색, 저장·다시 불러오기, 2차 검증, 깨진 글자, 표 칸 맞추기, 미리 보기	104	104	0	완료
통합 테스트	2026-09-24 ~ 09-28	기획서 테스트 시나리오 1 ~ 8 (등록 → 지출 입력 → 저장 → 조회 → 수정 → 삭제 → 재실행)	8	8	0	완료
시스템 테스트	2026-09-24 ~ 09-28	터미널에서 실제 실행 : 메뉴 전체 흐름, 잘못된 입력, 손상 파일 처리, '취소' 입력, 메뉴 번호 범위 오류	58	58	0	완료
시스템 테스트 (수정 후 재확인)	2026-09-28	한글 지우기 (readline 적용 후 3가지 경우), 공연 등록 중 한글 지우기, 방향키·이전 입력 불러오기, 화면 지우기 제어 문자 출력, 손상 파일 백업	8	8	0	완료

시험하면서 발견한 결함과 조치는 다음과 같다. 우회한 D-07을 뺀 나머지는 모두 수정한 뒤 자체 테스트를 다시 돌려 확인했다.

번호	발견한 문제	원인	조치	결과
D-01	좌석·예매처에 ♠가 출력되고, data.json이 중간에서 잘려 다음 실행 때 읽을 수 없음	백스페이스가 한글의 1바이트만 지워 깨진 바이트가 입력에 섞이고, 저장 중 오류가 나서 파일이 반만 쓰임	입력에서 깨진 글자 제거, 저장할 내용을 먼저 만들어 확인한 뒤 파일 열기, 저장 실패 안내	해결

번호	발견한 문제	원인	조치	결과
D-02	맥 터미널에서 한글을 지우면 빈칸이 생기고, 한 글자를 지우려면 백스페이스를 3번 눌러야 함	input()의 줄 편집을 운영체제가 바이트 단위로 처리	readline 적용	해결
D-03	메뉴를 넘어간 뒤 위로 스크롤하면 이전 화면이 남아 있음	보이는 화면만 지우는 코드(ESC[2J)만 사용	스크롤 기록 지우기(ESC[3J)를 마지막에 보내도록 순서 변경	해결
D-04	캐스팅이 3명 이상이면 표 칸보다 긴 글자를 그대로 자름	표 칸보다 긴 글자를 그대로 자름	칸을 넘으면 끝에 '..' 표시	해결
D-05	순위에 한 번도 안 본 공연이 0원으로 들어감	모든 공연을 순위에 넣음	관람 기록이 없는 공연은 순위에서 제외	해결
D-06	자체 테스트를 돌리면 실제 data.json이 바뀜	저장할 파일 이름이 함수를 만들 때 정해짐	저장할 때 파일 이름을 읽도록 수정, 테스트는 test_data.json 사용	해결
D-07	PyCharm 실행 창에서 한글이 한 박자씩 밀리고 Enter를 두 번 눌러야 넘어감	실행 창이 진짜 터미널이 아니라서 입력기가 조합 중인 글자를 늦게 넘김	README에 터미널 실행 권장과 PyCharm 설정 방법 안내	우회 (I-001)

6. 미완료·이슈 및 조치 계획

기획서에서 계획한 기능과 산출물 중 미완료 항목은 없다. 개발하면서 나온 이슈의 처리 결과와 남은 개선 과제는 다음과 같다.

ID	이슈/미완료 사항	영향도	조치 계획	완료 예정일	담당	상태
I-001	PyCharm 기본 실행 창에서는 readline과 화면 지우기가 동작하지 않음	하	실행 창 자체의 한계라 코드로는 고칠 수 없음. README.md에 터미널 실행 권장과 PyCharm 설정(Emulate terminal in output console) 안내, 깨진 글자 필터는 계속 유지	2026-09-28	이재성	우회

ID	이슈/미완료 사항	영향도	조치 계획	완료 예정일	담당	상태
I-002	콘솔 표에서 긴 공연명 캐스팅이 잘림	하	잘린 글자 끝에 '..'을 붙여 알리고, 상세 보기에서 전체 확인	2026-09-27	이재성	완료
I-003	그래프 같은 시각적 통계가 없음	하	기획 범위 밖의 개선 과제. 통계를 CSV로 내보내 스프레드시트에서 보는 기능 검토	미정	이재성	향후 과제

7. 운영 전환 계획

항목	계획 내용	담당	예정일	상태
배포	최종 소스를 GitHub 저장소(gwangeuk-log)에 올린다. 사용자는 폴더를 받아 추가 설치 없이 python3 main.py로 실행한다.	이재성	2026-09-29	완료
데이터 이관	새로 만든 프로그램이라 옮길 기존 데이터가 없다. 다른 컴퓨터로 옮길 때는 프로그램 폴더와 data.json을 같이 복사하면 된다.	사용자	-	해당 없음
사용자 교육	README.md의 '처음 사용할 때 따라하기'(공연 등록 → 관람 기록 추가 → 통계 확인)로 사용법을 안내한다.	이재성	2026-09-28	완료
운영 인수인계	소스코드, README.md, 설계서, 본 보고서를 발주자에게 전달한다.	이재성	2026-10-01	완료
장애 대응	README.md의 '문제가 생겼을 때' 표로 먼저 해결한다. data.json이 깨지면 원래 파일을 data_backup.json으로 남기고 새로 시작할 수 있다. 그래도 안 되면 GitHub 이슈나 개발자에게 알린다.	이재성	상시	완료

8. 산출물 목록

번호	산출물명	버전	파일/저장 위치	제출 여부	비고
1	기획 및 요구사항 정의서	-	기획 및 요구사항 정의서.pdf	제출	2026-09-09
2	소프트웨어 설계서	v1.0	문서/소프트웨어_설계서.pdf	제출	2026-09-25 수정
3	소스코드	최종	github.com/hello-g-i-t-h-u-b-world/gwangeuk-log	제출	모듈 7개
4	사용자 가이드 (운영 매뉴얼)	최종	README.md	제출	
5	시험 결과서	v1.0	본 보고서 5장, 부록 3	제출	자체 테스트 코드 test_calc.py
6	데이터 파일 샘플	최종	sample_data.json	제출	공연 5건, 관람 9건
7	소프트웨어 개발 완료 보고서	v1.0	본 문서	제출	

9. 종합 의견 및 완료 판단

기획서 1.2에서 세운 개발 목표 네 가지를 모두 달성했다. 기능 요구사항 FR-01 ~ FR-06과 예외 처리 EH-01 ~ EH-05를 모두 구현했고, 기획서의 테스트 시나리오 1 ~ 8과 자체 테스트 확인 항목 104개를 모두 통과했다. 파일 구성과 호출 방향도 설계서 3.1 구성도와 같다.

할인, 쿠폰, 수수료, 양도, MD가 겹치는 경우에도 계산 결과가 손으로 계산한 값과 같았다. 잘못된 입력이나 깨진 저장 파일에도 프로그램이 멈추지 않는다. 특히 한글 입력 때문에 저장 파일이 깨지던 문제를 바이트 단위까지 원인을 찾아 해결했다.

PyCharm 기본 실행 창에서는 한글 입력과 화면 지우기가 제대로 동작하지 않는다(6장 I-001). 실행 창 자체의 한계라 코드로는 고칠 수 없어서, README.md에 터미널에서 실행하는 방법과 PyCharm 설정을 안내해 두었다. 통계를 그래프로 보여주는 기능은 기획 범위 밖이라 향후 과제로 남겼다(6장 I-003). 또 기록이 아주 많아지면 저장할 때마다 파일 전체를 다시 쓰는 방식이 느려질 수 있지만, 개인이 쓰는 관람 기록 규모에서는 문제가 없다.

추가 설치 없이 `python3 main.py`로 바로 실행할 수 있고, README.md에 사용법과 문제 해결 방법을 정리했다. 샘플 데이터 파일까지 모든 산출물을 갖춰 발주자가 바로 사용할 수 있는 수준으로, 개발을 완료하였다.

[부록]

1. 설계서 변경 사항

설계서 v1.0과 비교해 기능, 모듈 구성, 데이터 구조는 바뀐 것이 없다. 다만 설계서에 자세히 적혀 있지 않던 부분을 구현하면서 다음과 같이 정했다.

항목	설계서	구현	이유
순위 대상과 같은 금액	공연별 총 순지출을 큰 순서대로 정렬	관람 기록이 없는 공연은 뺀다. 금액이 같으면 같은 순위로 매기고 다음 순위는 그 수만큼 건너뛴다 (1위, 1위, 3위).	한 번도 안 본 공연이 0원으로 순위에 들어가는 것이 어색함
한글 입력	(없음)	깨진 글자 필터, readline 적용	맥 터미널에서 한글을 지우면 깨진 바이트가 남음
화면 지우기	메뉴 이동 시 화면을 지우고 새로 그림	보이는 화면과 스크롤 기록까지 지움	위로 스크롤하면 이전 화면이 남음
표 출력	표로 정렬, 천 단위 콤마	한글을 2칸으로 세서 줄을 맞추고, 칸보다 긴 글자는 '.'으로 줄임	한글이 섞이면 줄이 어긋나고 긴 캐스팅이 잘림

2. 구현

2.1 자료 구조

프로그램은 시작할 때 data.json을 한 번 읽어서 파이썬 딕셔너리로 메모리에 올려 두고, 모든 기능이 이 딕셔너리 하나를 같이 쓴다. 공연(performances) 안에 관람 기록(viewings)이, 관람 기록 안에 지출 정보(expense)가 들어가는 구조이다. 아래는 발표 시연에 쓴 실제 데이터의 일부이다.

data.json

```
{
  "performances": [
    {
      "id": "P1", "title": "ASIA ARENA TOUR 2026 HORO",
      "type": "콘서트", "venue": "인스파이어 아레나",
      "viewings": [
        {
          "id": "V1", "date": "2026-09-20", "seat": "스탠딩석",
          "casting": ["Vaundy"],
          "expense": {
            "ticket_price": 170000, "discount_name": "조기예매할인", "discount_rate": 0.1,
            "coupons": [{ "name": "정부원쿠폰", "amount": 10000 },
                        { "name": "네이버 쿠폰", "amount": 3000 }],
            "booking_fee": 2000, "booking_site": "NOL",
            "trade_type": "직접구매",
            "actual_price": 142000,
            "transfer_income": 0,
            "md_items": [{ "name": "포스터", "amount": 15000 }]
          }
        }
      ]
    }
  ]
}
```

함수는 역할에 따라 다음과 같이 나뉘었다. 등록·수정 함수는 모두 (결과, 메시지, 저장 성공 여부)를 돌려주고, 결과가 None이면 검증에 실패한 것이라 메시지에 이유가 들어 있다. 화면 쪽은 이 약속 하나만 알면 된다.

파일	주요 함수	역할
cli.py	start(), performance_menu(), viewing_menu(), statistics_menu(), search_screen()	메뉴 화면
cli.py	get_input(), input_number(), input_date()	입력 받기와 1차 검사. '취소'를 입력하면 이전 메뉴로 돌아감
cli.py	print_header(), clear_screen(), fill(), cut_text()	화면 지우기, 제목 출력, 표 칸 맞추기
service.py	add_viewing(), update_viewing(), delete_viewing() 등	2차 검증 → 계산 → 메모리 수정 → 저장 순서로 처리
service.py	preview_expense(), search_performances()	저장 전 계산 결과 미리 보기, 검색
expense.py	calculate_expense(), calculate_actual_price(), calculate_net_expense()	지출 계산

파일	주요 함수	역할
statistics.py	make_summary(), monthly_statistics(), performance_analysis(), ranking()	통계 집계와 순위
repository.py	load_data(), save_data(), start_new_file()	불러오기(정상 / 새파일 / 오류), 저장, 손상 파일 백업
input_utils.py	check_number(), check_date(), check_viewing(), remove_broken_text()	1차 형식 검사, 2차 검증(모순 검사), 깨진 글자 제거

2.2 핵심 함수

(1) 지출 계산 : expense.py

관람 기록 한 건의 실제 구매금액과 순지출을 계산한다. 양도받은 티켓은 양도자에게 준 돈을 그대로 쓰고, 그 밖에는 정가에서 할인과 쿠폰을 빼고 수수료를 더한다. 쿠폰 집계에서는 '예매수수료 면제 쿠폰'을 빼고, 면제 쿠폰이 있으면 수수료를 0원으로 본다. 쿠폰이 너무 커서 음수가 되면 0원으로 바꾸고 안내 문구를 함께 돌려준다.

expense.py

```
def calculate_actual_price(expense):
    # 양도받은 티켓은 양도자에게 준 돈이 그대로 실제 구매금액이다
    if expense["trade_type"] == "양도받음":
        return expense["ticket_price"], ""

    price = expense["ticket_price"]
    price = price - get_discount_price(expense)
    price = price - get_coupon_total(expense)
    price = price + get_booking_fee(expense)

    # 쿠폰이 너무 커서 금액이 마이너스가 되면 0원으로 바꾼다
    if price < 0:
        return 0, "할인·쿠폰 금액이 결제 금액보다 커서 실제 구매금액을 0원으로 처리했습니다."

    return price, ""

def calculate_net_expense(expense):
    net = expense["actual_price"]
    net = net + get_md_total(expense)
    net = net - expense["transfer_income"]
    return net
```

설계서 예시로 계산하면 다음과 같다.

항목	금액	누계
티켓 정가	170,000원	170,000원
할인 : 조기예매할인 10%	- 17,000원	153,000원
쿠폰 : 정부원쿠폰, 네이버 쿠폰	- 13,000원	140,000원
예매 수수료	+ 2,000원	142,000원 (실제 구매금액)
MD : 포스터	+ 15,000원	157,000원 (순지출)

(2) 저장 전 2차 검증과 처리 순서 : service.py, input_utils.py

화면에서 입력을 받을 때 1차로 형식을 검사하지만, 저장 직전에 service.py가 완성된 값을 한 번 더 검사한다. 검증에 실패하면 메모리의 data를 건드리지 않고 이유만 돌려주기 때문에, 잘못된 값이 저장될 일이 없다. 순서는 항상 검증 -> 계산 -> 메모리 수정 -> 저장이다.

service.py

```
def add_viewing(data, performance_id, date, seat, casting, expense_data):
    ...
    # 1. 2차 검증 : 필수값이랑 서로 안 맞는 값이 없는지 확인
    error = input_utils.check_viewing(date, expense_data)
    if error != "":
        return None, error, False

    # 2. 금액 계산은 expense.py 가 한다
    expense_data, message = expense.calculate_expense(expense_data)
    ...
    performance["viewings"].append(viewing)
    saved = repository.save_data(data)
    return viewing, message, saved
```

2차 검증에서는 필수값과 금액 범위뿐 아니라 항목끼리 서로 맞지 않는 경우도 막는다.

input_utils.py - check_viewing()

```
# 6. 항목 간 모순
# 양도받은 티켓은 양도자한테 준 돈만 쓰니까 할인, 쿠폰, 수수료가 있으면 안 맞는다
if expense["trade_type"] == "양도받음":
    if rate > 0 or len(expense["coupons"]) > 0 or expense["booking_fee"] > 0:
        return "양도받은 티켓에는 할인, 쿠폰, 예매 수수료를 넣을 수 없습니다."
# 양도하지 않았는데 양도하고 받은 돈이 있으면 안 맞는다
if expense["trade_type"] != "양도함" and expense["transfer_income"] > 0:
    return "양도함이 아닌데 양도 금액이 들어가 있습니다."
```

(3) 순위 번호 매기기 : statistics.py

공연별 총 순지출을 큰 순서로 정렬한 뒤 순위 번호를 붙인다. 앞 공연과 금액이 같으면 같은 순위를 주고, 다르면 '줄 선 위치 + 1'을 준다. 그래서 1위가 셋이면 다음은 4위가 된다. 관람 기록이 없는 공연은 정렬 전에 미리 뺀다.

statistics.py - ranking()

```
# 총 순지출이 큰 순서대로 정렬
rank_list.sort(key=get_net, reverse=True)

# 순위 번호 붙이기 (금액이 같으면 같은 순위로 한다)
for i in range(len(rank_list)):
    if i > 0 and rank_list[i]["net"] == rank_list[i - 1]["net"]:
        rank_list[i]["rank"] = rank_list[i - 1]["rank"]
    else:
        rank_list[i]["rank"] = i + 1
```

(4) 한글이 섞인 표 맞추기 : cli.py

터미널에서 한글은 두 칸, 영문과 숫자는 한 칸을 차지한다. 파이썬의 len()은 글자 수만 세기 때문에 그대로 쓰면 한글이 섞인 표의 줄이 어긋난다. 그래서 화면에서 차지하는 칸 수를 따로 세고, 칸보다 긴 글자는 잘린 것을 알 수 있게 '..'을 붙인다.

cli.py

```
def get_width(text):
    width = 0
    for ch in text:
        if ord(ch) > 127:
            width = width + 2
        else:
            width = width + 1
    return width

def cut_text(text, size):
    if get_width(text) <= size:
        return text

    # ".." 들어갈 두 칸을 남기고 자른다
    while get_width(text) > size - 2:
        text = text[0:len(text) - 1]

    # 잘린 끝에 심표나 띄어쓰기가 남으면 "태양, 지디, .." 처럼 지저분해서 지운다
    while text.endswith(",") or text.endswith(" "):
        text = text[0:len(text) - 1]

    return text + ".."
```

(5) 한글 입력 문제 해결 : input_utils.py, repository.py, main.py

맥 터미널에서 한글을 백스페이스로 지우면 한 글자(3바이트) 중 1바이트만 지워지고, 남은 바이트가 깨진 글자가 되어 저장 파일까지 깨졌다. 세 단계로 해결했다.

① **입력에서 걸러내기** : 파이썬은 글자가 되지 못한 바이트를 서로게이트 문자(0xD800 ~ 0xDFFF)로 받는다. 이 범위와 이미 로 바뀐 글자를 지운다.

input_utils.py

```
def remove_broken_text(text):
    result = ""
    removed = False
    for ch in text:
        code = ord(ch)
        if code >= 0xD800 and code <= 0xDFFF:      # 깨진 바이트
            removed = True
            continue
        if code == 0xFFFD:                          # 이미  로 바뀌어서 들어온 경우
            removed = True
            continue
        result = result + ch
    return result, removed
```

② **확인한 뒤에 저장하기** : open(파일, "w")를 하는 순간 원래 내용이 지워진다. 그래서 저장할 내용을 먼저 문자열로 만들고 UTF-8로 쓸 수 있는지 확인한 다음에만 파일을 연다. 실패해도 기존 data.json은 그대로 남는다.

repository.py

```
def save_data(data):
    # 1. 저장할 내용을 먼저 글자로 다 만들어 본다
    # 여기서 실패하면 data.json 은 아직 열지도 않았으니까 그대로 남는다
    try:
        text = json.dumps(data, ensure_ascii=False, indent=2)
        text.encode("utf-8")      # 파일에 쓸 수 있는 글자인지 미리 확인
    except:
        return False

    # 2. 문제가 없으면 그때 파일에 쓴다
    try:
        f = open(DATA_FILE, "w", encoding="utf-8")
        f.write(text)
        f.close()
        return True
    except:
        return False
```

③ **처음부터 제대로 입력받기** : readline 모듈을 불러오면 input()의 줄 편집을 readline이 맡아서 한글이 한 글자씩 지워진다. 윈도우에는 이 모듈이 없어서 try로 감쌌고, 깨진 바이트가 들어와도 멈추지 않도록 surrogateescape도 설정했다.

main.py

```
try:
    import readline
except:
    pass

def main():
    # 0. 입력에 깨진 글자가 섞여 들어와도 프로그램이 멈추지 않게 한다
    sys.stdin.reconfigure(errors="surrogateescape")
    ...
```

2.3 개발하면서 신경 쓴 부분

저장 전에 계산 결과 보여주기 : 지출을 다 입력하면 바로 저장하지 않고 계산 결과를 먼저 보여준 뒤 '저장하시겠습니까? (y/n)'를 묻는다. preview_expense()는 지출 정보를 복사해서 계산하기 때문에 n을 골라도 원래 값이 바뀌지 않는다.

1차·2차 검증 : 입력할 때 바로 알려 주는 검사와 저장 직전의 검사를 나눴다. 화면을 거치지 않는 호출(자체 테스트 등)에서도 잘못된 값이 저장되지 않는다.

한 방향 호출 구조 : 계산·통계·저장·검증 파일은 다른 파일을 부르지 않아서 하나를 고쳐도 다른 곳에 영향이 없다.

멈추지 않는 입력 : 어디서든 '취소'를 입력하면 이전 메뉴로 돌아가고, Ctrl + C로 입력이 끊겨도 프로그램이 죽지 않고 취소로 처리한다.

증상과 원인을 나눠서 해결 : 한글 입력 문제를 입력 필터와 저장 순서로 먼저 막고(증상), readline으로 원인 자체를 없앴다. readline이 동작하지 않는 환경을 위해 필터는 남겨 두었다.

2.4 실행 화면

아래 화면은 모두 macOS 터미널에서 실제로 실행해서 캡처한 것이다.

(1) 공연 등록과 관람 기록 추가

```
=====
관극로그 : 공연 관람 가계부
메인 메뉴
=====
등록된 공연 1건 / 관람 기록 0건
=====
1. 공연 관리
2. 관람 기록 관리
3. 통계
4. 검색
0. 종료
=====

메뉴 번호 (0 ~ 4): 1
```

그림 1. 메인 메뉴 : 등록된 공연과 관람 기록 수를 보여주고 번호로 메뉴를 고른다

```
=====
관극로그 > 공연 관리 > 공연 등록
=====
('취소' 라고 입력하면 이전 메뉴로 돌아갑니다)

공연명 : ASIA ARENA TOUR 2026 HORO
공연 종류 (예 : 콘서트 / 뮤지컬) : 콘서트
공연장 : 인스파이어 아레나
=====

P1 등록 완료 : ASIA ARENA TOUR 2026 HORO
data.json 에 저장했습니다.

[Enter] 계속하기 ...
```

그림 2. 공연 등록 : 번호(P1)가 자동으로 붙고 바로 data.json에 저장된다

```
=====
관극로그 > 관람 기록 관리
=====
ID      공연명                  종류      공연장                  관람      순지출
-----
P1      ASIA ARENA TOUR 20..콘서트  인스파이어 아..      0회      0원
=====
선택할 공연 ID (예 : P1):
```

그림 3. 관람 기록 관리 : 기록을 추가할 공연 선택

관극로그 > 관람 기록 관리 > 계산 결과 확인

P1 ASIA ARENA TOUR 2026 HORO

관람일 : 2026-09-20

좌석 : 스탠딩석

캐스팅 : Vaundy

거래 유형 : 직접구매

예매처 : NOL

티켓정가170,000원

할인조기에매할인(10%)-17,000원

쿠폰정부원쿠폰-10,000원

쿠폰네이버쿠폰-3,000원

예매수수료+2,000원

실제구매금액142,000원

MD포스터+15,000원

순지출157,000원

저장하시겠습니까?(y/n): y

V1 등록 완료

data.json에 저장했습니다.

[Enter] 계속하기...

그림 4. 관람 기록 추가 : 저장 전에 계산 결과를 보여주고 저장 여부를 묻는다 (실제 구매금액 142,000원, 순지출 157,000원)

(2) 통계, 순위, 검색

관극로그 > 통계 > 월별 지출 통계

2026년 9월

관람일

공연명

티켓지출

MD

순지출

2026-09-19ASIA ARENA TOUR 2026..300,000원37,000원337,000원

2026-09-20ASIA ARENA TOUR 2026..142,000원15,000원157,000원

[2026년 9월 합계]

관람횟수

총티켓지출

총MD지출

총순지출

2회

442,000원

52,000원

494,000원

[Enter] 계속하기...

그림 5. 월별 지출 통계 (FR-03) : 2026년 9월, 2회 관람, 총 순지출 494,000원

그림 6. 공연별 지출 분석 (FR-04) : 관람 기록, 누적 합계, 회당 평균 247,000원

관극로그 > 통계 > 공연별 순위					
총 순지출이 큰 순서					
순위	ID	공연명	총 순지출	관람 횟수	
1위	P1	ASIA ARENA TOUR 2026 HORO	494,000원	2회	
2위	P2	싸이 흠뻑쇼 SUMMER SWAG..	187,000원	1회	
[Enter] 계속하기 ...					

그림 7. 공연별 순위 (FR-05) : 총 순지출이 큰 순서

관극로그 > 검색	
FR-06 공연 검색 및 필터링	
무엇으로 검색할지 골라주세요. (대소문자는 구분하지 않습니다)	
1. 공연명	
2. 공연장	
3. 캐스팅	
('취소'라고 입력하면 이전 메뉴로 돌아갑니다)	
검색 기준 : 1	
검색어 : HORO	

그림 8. 검색 (FR-06) : 검색 기준(공연명)과 검색어 입력

```
=====
관극로그 > 검색 > 검색 결과
공연명 : HORO
=====

공연 1건을 찾았습니다.

[P1] ASIA ARENA TOUR 2026 HORO (콘서트 / 인스파이어 아레나)
ID   관람일       좌석       캐스팅       순지출
-----
V1   2026-09-20   스탠딩석   Vaundy       157,000원
V2   2026-09-19   스탠딩석   Vaundy       337,000원

[Enter] 계속하기 ...
```

그림 9. 검색 결과 : 찾은 공연과 그 공연의 관람 기록

(3) 예외 처리

```
공연명 :
! 공연명은 (는) 필수 항목입니다. 다시 입력해주세요.
```

그림 10. 필수 항목(공연명)을 비웠을 때 (EH-02)

```
관람일 (YYYY-MM-DD) [2026-09-20]: 20260920
! 관람일은 (는) YYYY-MM-DD 형식으로 입력해주세요. (예 : 2026-08-21)
```

그림 11. 날짜 형식이 틀렸을 때 (EH-01)

```
티켓 정가 [170,000원]: 십만원
! 티켓 정가는 (는) 숫자만 입력해주세요. (예 : 170000)
```

그림 12. 금액에 글자를 넣었을 때 (EH-01)

```
=====
관극로그 > 공연 관리 > 공연 삭제
삭제 확인
=====

ID       : P1
공연명   : ASIA ARENA TOUR 2026 HORO
공연장   : 인스파이어 아레나
관람 기록 : 2건 (같이 삭제됩니다)
=====

정말 삭제하시겠습니까? (y/n): n
! 삭제를 취소했습니다.

[Enter] 계속하기 ...
```

그림 13. 삭제 확인에서 n을 골랐을 때 (EH-04) : 지울 내용을 보여주고, n이면 삭제하지 않는다

3. 결과 분석 및 검토

3.1 실행 결과로 확인한 것

샘플 데이터(sample_data.json)를 data.json으로 복사해 실행한 결과를 손으로 계산한 값과 비교했다.

확인 항목	손으로 계산한 값	프로그램 결과	일치
V1 할인율 (엔체인, 프리뷰 50%)	$66,000 - 33,000 + 2,000 = 35,000$ 원, 순지출 $35,000 + 10,000 = 45,000$ 원	35,000원 / 45,000원	○
V3 양도함 (싸이흠백쇼)	실제 구매 187,000 - 양도 금액 185,000 = 2,000원	2,000원	○
V5 쿠폰 2장 (엘리자벳)	$180,000 - (10,000 + 5,000) + 2,000 =$ 167,000원, 순지출 + MD 28,000 = 195,000원	167,000원 / 195,000원	○
V6 양도받음 (엘리자벳)	지급액 150,000원 그대로	150,000원	○
V9 수수료 면제 쿠폰 (곤 투모로우)	$130,000 - 26,000 - 3,000 + 0 =$ 101,000원, + 25,000 = 126,000원	101,000원 / 126,000원	○
2026년 9월 총 순지출	$150,000 + 69,500 + 259,000 + 126,000$ = 604,500원 (4회)	604,500원, 4회	○
P3 회당 평균	$345,000 \div 2\text{회} = 172,500$ 원	172,500원	○
순위	P3 345,000 > P5 259,000 > P4 195,500 > P2 189,000 > P1 93,200	1 ~ 5위 같은 순서 (그림 14)	○

=====				
관극로그 > 통계 > 공연별 순위				
총 순지출이 큰 순서				
=====				
순위	ID	공연명	총 순지출	관람 횟수

1위	P3	엘리자벳	345,000원	2회
2위	P5	Vaundy ASIA ARENA TOUR 2..	259,000원	1회
3위	P4	곤 투 모 로 우	195,500원	2회
4위	P2	싸이흠백쇼 SUMMER SWAG 2..	189,000원	2회
5위	P1	엔체인	93,200원	2회
[Enter] 계속하기 ...■				

그림 14. 샘플 데이터(sample_data.json)로 실행한 공연별 순위 (FR-05)

설계서 예시와 샘플 데이터 모두 손계산과 같게 나왔다. 계산식이 expense.py 한 곳에만 있고 통계는 저장된 계산 결과를 더하기만 하기 때문에, 할인·쿠폰·수수료·양도·MD가 섞여도 화면마다 금액이 달라지는 일이 없다.

3.2 검토했거나 앞으로 시도할 기술

임시 파일에 먼저 쓰고 바꾸기 (os.replace) : 저장 도중 전원이 꺼지는 경우까지 막을 수 있어 검토했다. 하지만 이번에 문제가 된 상황(깨진 글자)은 저장할 내용을 미리 만들어 확인하는 방법으로도 막을 수 있고, 원리를 직접 설명할 수 없어서 선택하지 않았다. 데이터가 더 중요해지면 적용해 볼 거 같다.

데이터베이스 : 기록이 아주 많아지면 JSON 파일 전체를 다시 쓰는 대신 SQLite 같은 데이터베이스를 쓸 것이다.

통계 시각화 : 월별 지출을 CSV로 내보내 스프레드시트에서 그래프로 보는 기능을 추가해 보면 좋을 거 같다.

3.3 코드 평가

평가 측면	내용
구조의 명쾌함	설계서 3.1대로 7개 파일로 나누고 호출을 한 방향으로 정리했다. 계산·통계·저장·검증 파일은 다른 파일을 부르지 않아서 순위 규칙을 바꿀 때 statistics.py만 고치면 됐다.
시간 복잡도	관람 기록 전체를 N건, 공연 수를 P개라 하면 월별 통계, 검색, 번호 만들기는 전체를 한 번 훑어서 O(N), 순위는 공연별 합계 O(N)에 정렬 O(P log P)이다. 저장은 바뀔 때마다 파일 전체를 다시 써서 O(N)이지만, 개인 관람 기록 규모에서는 바로 끝난다.
공간 복잡도	data.json 전체를 메모리에 올려 두므로 O(N)이다. 관람 기록 한 건이 1KB 안팎이라 수천 건이어도 몇 MB 수준이다.
가독성	for·if와 딕셔너리만 쓰는 단순한 문법이고, 함수마다 AI가 주석을 달아 처음 보는 사람도 따라가기 쉽다. 다만 cli.py가 1,100줄이 넘어서 화면별로 파일을 나누면 더 읽기 쉬울 것 같다.

4. 자체 평가서

4.1 수행 과정과 결과에 대한 생각

이번 프로젝트는 내가 만들고 싶은 것을 만드는 것이 아니라 다른 사람이 요구한 프로그램을 만드는 첫 작업이었다. 공연 예매나 MD 구매에 대한 배경지식이 부족해서, 요구서를 처음 봤을 때 어떻게 만들어야 할지 바로 떠오르지 않아 많이 당황했다. 티켓을 어떤 식으로 예매하는지, 할인과 쿠폰, 예매 수수료, 양도가 어떻게 이루어지는지, 어떤 MD 품목들이 있는지를 공부하고 나서야 데이터 구조와 계산식을 정할 수 있었고, 마침내 프로그램을 완성할 수 있었다. 모르는 분야의 프로그램을 요구받으면 그 분야에 대한 공부도 필요하다는 것을 느꼈다.

설계서를 먼저 써 둔 것이 큰 도움이 됐다. 구현하다가 헛갈릴 때마다 설계서를 기준으로 정할 수 있었고, 기획서와 설계서의 내용이 다를 때는 나중에 쓴 설계서를 따르기로 정해 두니 판단이 쉬웠다.

한글 입력 문제는 자체 테스트로는 보이지 않았고, 사용자의 입장에서 직접 여러 번 입력해 보고 나서야 찾을 수 있었다. 테스트 코드가 모두 통과해도 직접 써 봐야 보이는 문제가 있다는 것을 알게 됐다.

구현과 오류 수정에는 AI를 많이 활용했다. 다만 결과를 그대로 쓰지 않고, 코드가 어떤 원리로 동작하는지 설명을 듣고 이해한 다음에 반영하려고 했다. 예를 들어 저장 방식은 처음 제안받은 임시 파일 방식이 잘 이해되지 않아서, 원리를 직접 설명할 수 있는 더 단순한 방식으로 바꿨다.

4.2 수행 수준 자체 평가

평가 항목	점수	근거
요구사항 구현	5 / 5	FR-01 ~ FR-06, EH-01 ~ EH-05를 모두 구현했다.
계산 정확성	5 / 5	손계산과 모두 일치했고 자체 테스트 확인 항목 104개를 통과했다.
안정성	5 / 5	잘못된 입력, 깨진 저장 파일, 한글 입력 문제에도 프로그램이 멈추지 않는다.
코드 품질	4 / 5	설계대로 구조를 나눴지만 cli.py가 길고 예외를 한꺼번에 잡는 곳이 남아 있다.
문서화	5 / 5	README.md, 주석, 실제 공연 정보로 만든 샘플 데이터 파일까지 갖췄다.
일정 관리	3 / 5	기획서의 주차 계획보다 구현이 늦게 시작돼 테스트와 수정이 마지막 주에 몰렸다.
종합	27 / 30	계획한 기능과 목표, 산출물을 모두 달성했다. 코드 구조와 일정 관리는 더 다듬고 싶다.

4.3 새로 습득한 기술

한글 인코딩과 깨진 바이트 처리 (가장 크게 배운 것)

글자가 파일에는 바이트로 저장되고, 한글은 UTF-8에서 한 글자가 3바이트라는 것을 처음 제대로 알게 됐다. 파이썬은 깨진 바이트를 surrogateescape 설정으로 \udc80 ~ \udcff 범위의 서로게이트 문자에 담아 두는데, 이 문자는 UTF-8로 파일에 쓸 수 없어서 저장 오류가 난다. 또 open(파일, "w")로 여는 순간 원래 내용이 지워지기 때문에, 저장할 내용을 먼저 만들어 확인한 뒤에 파일을 열어야 한다는 것도 배웠다.

readline과 터미널의 줄 편집

input()은 기본으로 운영체제의 줄 편집을 쓰고, macOS 터미널은 백스페이스를 바이트 단위로 처리한다. readline 모듈은 import만 해도 input()의 입력을 대신 맡아서 한글이 글자 단위로 지워지고, 방향키로 커서를 옮기거나 이전 입력을 불러올 수도 있다.

ANSI 제어 문자로 화면 지우기

ESC[H(커서를 맨 위로), ESC[2J(보이는 화면 지우기), ESC[3J(스크롤 기록 지우기)로 화면을 지울 수 있다. 맥 터미널은 2J를 할 때 화면을 스크롤 기록으로 밀어 올리기 때문에 3J를 마지막에 보내야 다 지워진다.

5. 개발 일지

GitHub 저장소 : <https://github.com/hello-g-i-t-h-u-b-world/gwangeuk-log>

날짜	개발 내용	GitHub 커밋 (시각)
2026-09-07	발주자(김가은)에게 제안요청서를 받아 요구사항을 파악했다.	-
2026-09-09	기획 및 요구사항 정의서 작성 (FR-01 ~ FR-06, EH-01 ~ EH-05, 테스트 시나리오 1 ~ 8, 개발 일정)	-
2026-09-13	소프트웨어 설계서 v0.1 작성 (모듈 7개, 구성도, 시퀀스 다이어그램, 데이터 구조, 화면 설계)	-
2026-09-17	구현 1차 시작 : FR-01, FR-02, JSON 저장, 지출 계산, 자체 테스트 작성	- (저장소 만들기 전 로컬 작업)
2026-09-22	GitHub 저장소를 만들고 PyCharm 프로젝트를 올렸다.	Initial commit (21:51)
2026-09-25	설계서 v1.0 수정. 1차 구현(FR-01, FR-02, JSON 저장)을 올렸다.	FR-01, FR-02, json 구현 (13:00)
2026-09-26	구현 2차 : FR-03 ~ FR-06 (월별 통계, 공연별 분석, 순위, 검색). 순위에서 관람 기록이 없는 공연을 뺐다.	FR-03~FR-06 구현 (20:03, 20:04) statistics.py 수정 (21:00)

날짜	개발 내용	GitHub 커밋 (시각)
2026-09-27	PyCharm 실행 창 의 한글 입력 밀림 원인 확인. 좌석·예매처의 ❖ 출력과 data.json 손상 원인 분석 → 깨진 글자 필터, 저장 순서 변경, 저장 실패 안내. 긴 캐스팅 '.' 처리. 설계서 기준 전체 기능 점검(터미널 실행 검사).	09-29 최종 수정 커밋에 포함
2026-09-28	설계서 점검 결과 반영(모듈 호출 관계, 저장 전 계산 결과 확인 y/n), README.md 작성. 화면 지우기(스크롤 기록) 수정, readline 적용.	09-29 최종 수정 커밋에 포함
2026-09-29	9월 27일 ~ 28일 수정 내용을 커밋하고, 저장소에 올라간 PyCharm 설정 폴더(.idea)를 뺐다. 실제 2026년 공연 정보를 조사해 샘플 데이터 파일(sample_data.json)을 만들고 README.md에 사용 방법을 추가했다.	최종 수정 (10:07) 불필요한 .idea 폴더 제거 (10:15) 데이터 샘플 및 테스트 파일 업로드 (11:46)

※ 9월 17일의 구현 1차는 GitHub 저장소를 만들기 전에 로컬에서 작업했다. 커밋 시각은 저장소 기록 그대로이다.

6. 참고 문헌

[1] Python Software Foundation, "readline — GNU readline interface,"

<https://docs.python.org/3/library/readline.html>.

[2] Python Software Foundation, "json — JSON encoder and decoder,"

<https://docs.python.org/3/library/json.html>.

AI 활용 내역

AI 종류	사용 단계	질문 내용	응답 내용
GPT-5.6 Luna	설계	기획서를 바탕으로 기능·모듈을 나누고 모듈 이름과 역할을 알려 달라고 요청	모듈 구성 제안. 이름과 역할 일부를 수정해 7개 모듈로 확정 (설계서 부록 B)
GPT-5.6 Luna	설계	설계한 모듈을 바탕으로 시스템 구성도, 시퀀스 다이어그램, 예상 화면을 그려 달라고 요청	구성도, 시퀀스 다이어그램, 화면 예시 그림 (설계서 3.1, 4.3, 6.2)
Claude (Claude Code)	구현	기획서의 관리 계획대로 FR-01·FR-02와 JSON 저장을 구현해 달라고 요청	7개 모듈 코드 초안과 자체 테스트 코드
Claude (Claude Code)	구현	FR-03 ~ FR-06 구현 요청, 관람 기록이 없는 공연의 순위 처리 질문	통계·순위·검색 코드, 기록 없는 공연을 순위에서 빼도록 수정
Claude (Claude Code)	오류 수정	좌석·예매처에 ❖가 나오고 캐스팅이 잘리는 문제, Enter를 두 번 눌러야 넘어가는 문제 질문	UTF-8 바이트와 서로게이트 원인 설명, 입력 필터·저장 방식·저장 결과 안내 수정, 긴 글자 '..' 처리, 코드 한 줄씩 원리 설명
Claude (Claude Code)	문서	README.md 작성 요청	실행 방법, 메뉴 구조, 계산식, 문제 해결 표 작성
Claude (Claude Code)	오류 수정	스크롤하면 이전 화면이 남는 문제, 맥 터미널에서 한글을 지우면 빈칸이 생기는 문제 질문	화면 지우기 순서(ESC[3J) 수정, readline 적용과 동작 원리 설명